Workshop 2: DeepConvLSTM

You will run a systematic ablation study of DeepConvLSTM, a deep network for recognizing human activities from wearable sensors. You have

The task. The OPPORTUNITY dataset records people doing morning routines in a kitchen while wearing 113 sensor channels (accelerometers, gyroscopes, magnetometers) sampled at 30 Hz. The model sees 0.8 s windows of these signals and predicts one of 18 classes: 17 gestures (open the fridge, drink from a cup, …) or Null (no gesture). The model combines 4 convolutional layers, which learn features from the raw signals, with 2 LSTM layers, which model how those features change over time.

What an ablation is. An ablation changes exactly one part of a system and measures the effect, while everything else stays the same. This is how researchers find out which design choices actually matter, and which were arbitrary.

File	What it does
download_data	Downloads OPPORTUNITY from the UCI repository (300 MB)
preprocess	Selects the 113 channels, fills missing values, normalizes to [0, 1]
data	Cuts recordings in windows, defines sensor subsets, removes Null class on request
models	DeepConvLSTM and BaselineCNN
train	Trains one model, picks best epoch on validation, evaluates on test, saves everything
evaluate	Reloads a saved model and evaluates it on any split
summarize	Collects all finished runs into two CSV tables

Setup on your own machine:

```
pip install -r requirements.txt
python download_data.py
python preprocess.py
python train.py
```

Teams. Split Parts between team members, but every member must understand every part: the report is written together and anyone may be asked to explain any result. And all codes and reports should be realized on GitHub.

Part 0: Reproduce the reference model (10%)

Run: Train the reference model

Part A: Is the LSTM what helps? (10%)

Question. The paper's central claim is that adding recurrent layers on top of convolutions improves recognition, because LSTM models how features unfold over time. It supports this with one comparison: DeepConvLSTM vs. a CNN with dense layers instead of LSTMs.

Runs: Test the claim with **2** other models. (CNN, RNN GRU)

Variant	Options	What changes
A_cnn	--model cnn	2 dense layers of 128 units instead of the LSTMs
A_gru	--rnn gru	GRU cells instead of LSTM cells

Part B: Do the convolutions matter? (10%)

Question. The paper argues that convolutions learn useful features directly from raw signals, so no hand-crafted features are needed. Its Figure 8 shows weighted F1 rising with the number of conv layers on gestures with Null. It never tests zero conv layers, nor the kernel size or the number of filters.

Runs : Run the code **2** times, with 1 and 2 conv layers

Variant	Options	What changes
B_conv0	--n-conv 0	No convolutions: the LSTM reads the 113 raw channels
B_conv1	--n-conv 1	1 conv layer
B_conv2	--n-conv 2	2 conv layers

Part C: How long should the window be? (10%)

Question. The window decides how much of the past the model sees before it predicts. The paper tried windows of 400, 500, 1400 and 2750 ms and reported per-gesture F1 in Figure 7. It found little change for most gestures, but short drawer gestures did better when they fit inside the window.

Runs: Use **2** different window size (0.67s and 2 s) to run the experiments

Variant	Options	Window length
C_w20	--window 20	0.67 s, the shortest possible with 4 conv layers
C_w42	--window 42	1.4 s, as in the paper
C_w60	--window 60	2.0 s

Part D: How much regularization is needed? (10%)

Question. The model has about 4 million parameters but only tens of thousands of training windows (the first log line gives the exact number), so it can overfit. The paper uses dropout 0.5 and nothing else. In one sentence of Section 5.4 it says that higher dropout made "no difference" and that learning and regularization settings matter less than the architecture, but it shows no numbers. You will check this.

Runs :**3** different runs for 1)no dropout, 2)no weight decay, 3)no augmentation

Part E: Which optimizer settings work best? (10%)

Question. The paper trains with RMSProp (learning rate 0.001, decay 0.9) on batches of 100. Section 3.3 says that after trying several optimizers, RMSProp "consistently offered the best results with the widest tolerance to the learning rate setting". No numbers are given, and batch size is not discussed.

Runs : Just **1** run : Add a learning-rate warm-up (e.g., a linear increase over the first 5 epochs) and test it with the learning rate that worked worst. Does warm-up make training more stable?

Part F: Does handling class imbalance help rare gestures? (10%)

Question. About 83% of the test windows are Null, while several gestures have fewer than 100 test windows. A reference run reaches about 0.89 weighted F1 but only about 0.60 macro F1; some drawer gestures stay below 0.40 F1. The paper does nothing about the imbalance and reports weighted F1, which Null dominates.

Runs : Just **1** run: implement Focal Loss and compare with the results with the paper